

DEPARTMENT OF COMPUTER SCIENCE
FINAL YEAR PROJECT PROPOSAL

NeuroCardia

Edge-AI ECG Arrhythmia Classification and Real-Time Telemetry Platform

A Privacy-Preserving Mobile and Web Platform for On-Device ECG Analysis and Remote Event Monitoring

Project Team Members	Waleed Ali Khan Ayinaa Haseeb Khan Hishaam Khan
Domain	Healthcare AI & Edge Computing
Project Duration	One Academic Year (Two Semesters)
Project Type	Artificial Intelligence, Mobile Computing & Full-Stack Development
Proposed Supervisor	Department Faculty

This proposal has been refined from the original submission to establish a scope that is realistically achievable within a two-semester final-year project while preserving the project's core technical contribution.

1. ABSTRACT

Cardiac arrhythmias are abnormalities in the electrical activity of the heart that may occur intermittently and can therefore remain undetected during short clinical examinations. Continuous electrocardiogram (ECG) monitoring provides an opportunity to observe cardiac activity over extended periods, but conventional monitoring approaches typically depend on continuous network connectivity and centralized processing, which raises latency, bandwidth, and privacy concerns.

NeuroCardia is a research prototype that performs automated ECG beat classification directly on a mobile device and reports detected events, rather than raw signal, to a web-based monitoring interface. The system comprises a Flutter mobile application performing on-device signal preprocessing and inference, a FastAPI backend responsible for event ingestion and real-time delivery, and a Next.js dashboard for live and historical event review. The classification model is trained on the MIT-BIH Arrhythmia Database using the standard AAMI five-class beat grouping, with an explicit class-imbalance mitigation strategy, and is optimized for mobile deployment through INT8 quantization and TensorFlow Lite / LiteRT export.

The project's contribution is the demonstrated, benchmarked pipeline connecting ECG machine learning to edge deployment and live telemetry — evaluated on classification quality (macro-F1, per-class recall), deployment efficiency (model size, on-device latency), and end-to-end event delivery latency. NeuroCardia is an academic decision-support prototype; it is not a certified medical device and does not replace clinical diagnosis.

2. PROBLEM STATEMENT

Cardiac arrhythmias can occur unpredictably and are frequently missed during brief clinical examinations. Ambulatory ECG monitoring addresses this by observing cardiac activity continuously, but conventional architectures that stream raw ECG data to a central server for processing introduce several avoidable problems:

- Continuous transmission of raw ECG waveform data increases bandwidth use and battery drain on the recording device.
- Cloud-dependent processing introduces round-trip latency and fails when connectivity is unavailable.
- Centralized transmission and storage of physiological data broadens the attack surface for a sensitive data category.
- Simple threshold-based monitoring produces frequent false alarms and offers limited automated interpretation of rhythm.
- Published ECG machine-learning research frequently reports classification accuracy without demonstrating that the resulting model can actually run within the memory, compute, and latency budget of a mobile device.

NeuroCardia addresses these problems by keeping ECG classification on-device and transmitting only detected events — not continuous raw signal — to a lightweight backend and monitoring dashboard, demonstrating the complete pipeline from signal processing through edge inference to event-level remote visibility.

3. OBJECTIVES

- Design and implement an ECG preprocessing pipeline covering noise/baseline removal, R-peak detection, and heartbeat segmentation.

- Train and compare two candidate architectures (1D-CNN baseline and 1D-ResNet) for five-class heartbeat classification, with an explicit strategy for class imbalance.
- Optimize the selected model for mobile deployment via INT8 quantization and TensorFlow Lite / LiteRT export, and benchmark the resulting size, latency, and resource usage on target hardware.
- Implement a Flutter mobile application that performs on-device inference, visualizes ECG waveforms, operates offline, and synchronizes detected events when connectivity is available.
- Implement a FastAPI backend and PostgreSQL schema for event ingestion, authentication, and real-time delivery via WebSocket.
- Implement a Next.js dashboard providing a live event feed and historical session view for authorized users.
- Evaluate the complete system on machine-learning performance (macro-F1, per-class recall), edge performance (latency, model size, memory), and end-to-end event-delivery latency under lab network conditions.

4. PROPOSED SOLUTION AND ARCHITECTURE

NeuroCardia implements a three-tier architecture. Unlike the original proposal, session data is scoped to a device/user account rather than a clinical patient record, and only classified events — not continuous raw ECG — cross the network boundary. This removes the need for a patient-identity data model while preserving the project's edge-to-cloud demonstration.

4.1 Mobile Edge Tier — Flutter Application

Performs ECG preprocessing, R-peak detection, and on-device inference using a quantized TFLite model. Visualizes waveforms in real time, stores session data locally using Isar for offline operation, and synchronizes detected events (not raw signal) once connectivity is restored.

4.2 Backend Tier — FastAPI

Provides token-based authentication, REST endpoints for session and event history, and a WebSocket channel for real-time event delivery to the dashboard. PostgreSQL is the sole persistence layer; Redis has been removed from the core scope as it provides no benefit at single-instance deployment scale (see Section 12).

4.3 Monitoring Tier — Next.js Dashboard

Displays a live event feed over WebSocket and a historical session view. Interactive waveform zoom is retained as a core feature; automated clinical time/amplitude measurement tools are deferred to future work (Section 13), as they constitute a separate clinical-tooling problem outside this project's scope.

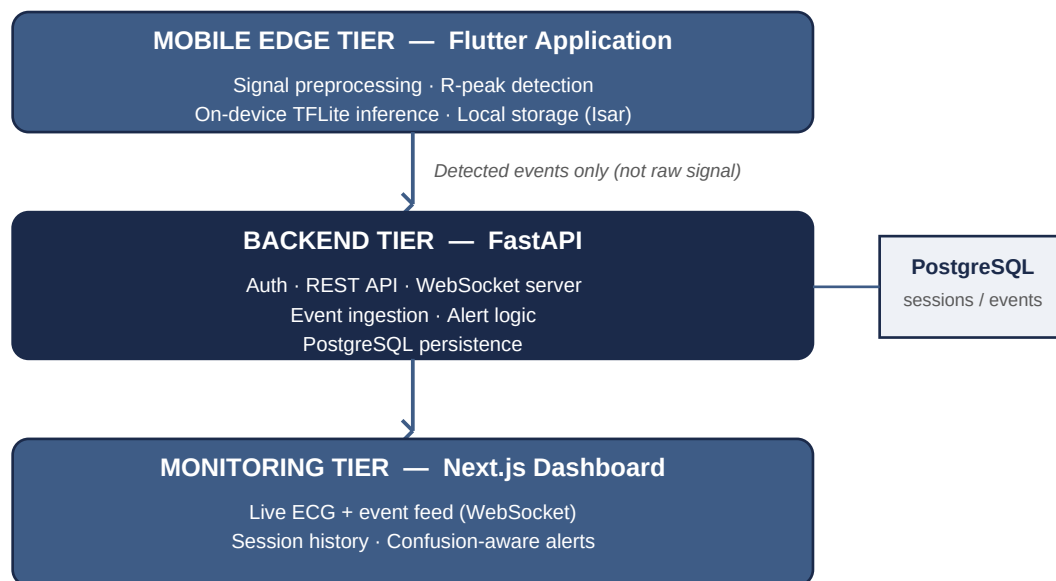


Figure 1. Refined three-tier architecture (Redis and patient-record management removed from core scope).

5. MACHINE LEARNING METHODOLOGY

5.1 Dataset

The MIT-BIH Arrhythmia Database (PhysioNet) is the sole dataset used for model development and evaluation: 48 half-hour, two-channel ambulatory ECG recordings sampled at 360 Hz with expert beat annotations. PTB-XL has been removed from scope — its 12-lead diagnostic-statement labels are not directly compatible with the AAMI beat-level classes used here, and reconciling the two labeling schemes is a separate research problem that would displace time from the core deployment pipeline. It may be noted as a possible extension for future work.

5.2 Signal Preprocessing

- Baseline wander and high-frequency noise removal via digital filtering.
- R-peak detection using an implementation of the Pan-Tompkins algorithm.
- Fixed-length heartbeat segmentation centered on detected R-peaks, window size to be finalized experimentally.
- Per-record amplitude normalization to reduce inter-recording variance.

5.3 Classification Task

The primary task is five-class heartbeat classification using the AAMI grouping shown in Table 1. The system performs beat-level classification only; it is not presented as a rhythm-level detector (e.g., it does not claim to detect ventricular fibrillation, which requires different data and methodology).

Class	Description
N	Normal / non-ectopic beat

Class	Description
S	Supraventricular ectopic beat
V	Ventricular ectopic beat
F	Fusion beat
Q	Unknown / paced / unclassifiable beat

Table 1. AAMI five-class beat grouping used for classification.

5.4 Class Imbalance Mitigation

MIT-BIH is heavily dominated by class N. Overall accuracy is therefore not a meaningful metric on its own (Section 7). The project will apply class-weighted loss during training as the primary mitigation, with oversampling of minority classes (S, F) evaluated as a secondary experiment if weighting alone is insufficient. This decision — and its effect on minority-class recall — will be reported explicitly rather than only stating final aggregate metrics.

5.5 Model Development

A two-stage comparison, not an open-ended architecture search, is used to keep the ML workload bounded:

- Stage 1 — Baseline: a lightweight 1D-CNN.
- Stage 2 — Residual network: a 1D-ResNet, evaluated against the baseline on macro-F1 and per-class recall.

Progression criterion: the ResNet stage is retained as the deployed model only if it improves macro-F1 over the CNN baseline by more than 2 percentage points; otherwise the simpler CNN is deployed, since model simplicity directly benefits edge latency and size. Squeeze-and-Excitation attention and BiLSTM temporal modeling are retained only as time-permitting ablation experiments, not as required deliverables — this is the single largest scope change from the original proposal and the one most responsible for making the ML component achievable within the semester timeline allocated to it (Section 14).

5.6 Evaluation

Primary metrics: macro F1-score, per-class precision/recall, and confusion matrix, with particular attention to the S and V minority classes. Secondary metrics: parameter count, exported model size, on-device inference latency, and memory footprint on the target mobile hardware. All figures reported will be those actually measured during development; no target accuracy is claimed in advance.



Figure 2. Model development-to-deployment pipeline.

6. EDGE-AI DEPLOYMENT

The selected model is exported and converted to TensorFlow Lite / LiteRT format, with INT8 post-training quantization evaluated for its effect on both model size and classification quality (a small accuracy trade-off is expected and will be reported, not hidden). The quantized model is integrated into the Flutter application for on-device inference. Inference latency, memory consumption, and model size are benchmarked on the actual target device used for development, not estimated theoretically.

7. FUNCTIONAL SCOPE

7.1 Mobile Application (Flutter)

- ECG waveform visualization and signal playback (recorded/simulated signals for development and demonstration).
- On-device preprocessing and classification with confidence display.
- Offline local storage (Isar) and background synchronization of detected events on reconnect.
- Configurable notification threshold for high-confidence abnormal events.

7.2 Backend (FastAPI + PostgreSQL)

- Token-based authentication and authorization.
- REST endpoints for session and event history.
- WebSocket channel for real-time event delivery to the dashboard.
- Event and session persistence in PostgreSQL; synchronization conflicts resolved with a timestamped last-write-wins policy, stated explicitly as the project's scoped approach rather than a full CRDT-style solution.

7.3 Monitoring Dashboard (Next.js)

- Live event feed over WebSocket.
- Session history list with basic filtering.
- Interactive ECG waveform zoom for reviewing a stored segment.

Automated clinical interval/amplitude measurement and session-level analytics are explicitly out of scope for the core deliverable (Section 13) — interactive inspection is retained, automated clinical interpretation is not.

8. TECHNOLOGY STACK

Component	Technology
Mobile application	Flutter / Dart
Edge inference runtime	LiteRT / TensorFlow Lite
ML development	Python, PyTorch
Signal processing	Python, NumPy, SciPy
Backend	FastAPI
Real-time communication	WebSockets
Database	PostgreSQL
Web dashboard	Next.js / React, Tailwind CSS
Waveform visualization	Canvas / D3.js
Local mobile storage	Isar
Authentication	Token-based authentication

Table 2. Final technology stack. Redis has been removed — it added an unresolved 'optional' dependency to the original proposal without a stated purpose at single-instance deployment scale.

9. SECURITY AND PRIVACY

ECG-derived data is treated as sensitive throughout the system, even though the project uses only public and simulated datasets and does not collect real patient data:

- Token-based authentication and route-level authorization on all backend endpoints.
- TLS-secured API and WebSocket communication.
- Raw ECG waveform data is not transmitted off-device except where explicitly needed for demonstration; only classified events are synced by default.
- Development and evaluation data (MIT-BIH, simulated signals) is kept separate from any account/session data used to demonstrate the system.
- Passwords are hashed using an established algorithm (e.g., bcrypt/argon2); no credentials are stored in plaintext.

NeuroCardia is an academic research prototype and decision-support demonstration. It is not a certified medical device, has not undergone clinical validation, and is not intended for unsupervised clinical use.

10. TESTING STRATEGY

Layer	Approach
ML model	Held-out test split from MIT-BIH; confusion matrix and per-class metrics reviewed after each training run.
Signal processing	Unit tests on R-peak detection against annotated beat positions.
Backend API	Unit tests for auth and event endpoints; integration test for the WebSocket event-delivery path.
Mobile application	Manual end-to-end test scripts covering offline capture, sync-on-reconnect, and on-device inference correctness against known recordings.
System-level	End-to-end demonstration run with simulated ECG input, measuring detection-to-dashboard latency.

Table 3. Testing responsibilities by system layer.

11. PROJECT SCOPE AND LIMITATIONS

11.1 Included

- ECG preprocessing, R-peak detection, heartbeat segmentation.
- Five-class beat classification with an explicit class-imbalance mitigation strategy.
- CNN vs. ResNet model comparison against a stated progression criterion.
- INT8 quantization and TFLite/LiteRT mobile deployment with measured benchmarks.
- Offline-capable Flutter application with on-device inference.
- FastAPI backend with authentication, REST API, and WebSocket telemetry.
- PostgreSQL persistence for sessions and events.
- Next.js live and historical monitoring dashboard.
- End-to-end latency and resource benchmarking.

11.2 Excluded from core scope

- Clinical patient-record management (sessions are device/account-scoped, not patient-identity-scoped).
- Redis or any secondary caching/state layer.
- PTB-XL or multi-dataset training.
- Automated clinical interval/amplitude measurement.
- Rhythm-level detection (e.g., ventricular fibrillation) as distinct from beat-level classification.
- Wearable sensor hardware integration.
- Clinical trials, regulatory certification, or hospital-system integration.
- Emergency-service contact or autonomous alerting to third parties.

SE-attention and BiLSTM modeling remain available as optional, time-permitting ablation experiments (Section 5.5) but are not required for project completion.

12. EXPECTED OUTCOMES

- A validated ECG preprocessing and R-peak detection pipeline.
- A trained beat classification model with a documented CNN-vs-ResNet comparison and class-imbalance mitigation.
- A quantized, mobile-deployed model with measured size, latency, and memory benchmarks.
- A working Flutter application performing offline-capable on-device inference and sync.
- A FastAPI/PostgreSQL backend with authentication and real-time WebSocket event delivery.
- A Next.js dashboard with live and historical event views.
- An end-to-end demonstration using public and simulated ECG signals, with measured detection-to-dashboard latency.
- A consolidated benchmark report (ML performance, edge efficiency, system latency) and final technical documentation.

13. PROJECT SIGNIFICANCE

NeuroCardia's contribution is integrative rather than purely algorithmic: rather than optimizing a classifier in isolation, the project demonstrates and benchmarks the complete pipeline from signal processing through edge deployment to real-time event delivery, and — unlike much published ECG-ML work — explicitly measures whether the resulting model is actually viable on constrained mobile hardware rather than assuming it. The explicit trade-off analysis between model complexity (CNN vs. ResNet, with quantization) and deployment efficiency is the project's central technical investigation.

14. TWO-SEMESTER IMPLEMENTATION PLAN

Month	Activities	Output
1	Literature review, requirements analysis, MIT-BIH acquisition and exploration.	Research & requirements document
2	Filtering, denoising, Pan-Tompkins R-peak detection, unit tests.	Preprocessing pipeline
3	Dataset preparation, class-imbalance strategy, baseline CNN.	Baseline model + evaluation report
4	ResNet implementation and comparison against progression criterion.	Model selection decision
5	Quantization, TFLite/LiteRT conversion and on-device benchmarking.	Edge-ready model + benchmark data
6	Flutter app shell, waveform visualization, model integration.	Mobile prototype with inference
7	Offline storage (Isar), notification logic, sync-on-reconnect.	Offline-capable mobile app
8	FastAPI backend: auth, REST endpoints, PostgreSQL schema.	Backend prototype
9	WebSocket telemetry, mobile-backend integration.	Real-time event pipeline
10	Next.js dashboard: live feed, history view, waveform zoom.	Web dashboard
11	Full system integration, end-to-end and latency testing, optional ablations if time permits.	Integrated, tested system
12	Benchmarking consolidation, documentation, final report and demonstration.	Final report, benchmark report, demo

Table 4. Revised month-by-month plan. Optional ablation experiments (SE-attention, BiLSTM) are scheduled only within Month 11's contingency, not on the critical path.

15. RISK MANAGEMENT

Risk	Likelihood	Mitigation
Minority classes (S, F) fail to reach usable recall despite mitigation.	Medium	Report honestly; treat as a discussed limitation rather than concealing via aggregate accuracy.
TFLite/LiteRT conversion or quantization introduces unexpected accuracy loss or conversion errors.	Medium	Budget dedicated Month 5 time; fall back to FP16 quantization if INT8 degrades accuracy unacceptably.
Offline sync introduces data-loss or duplication bugs.	Medium	Scope to a simple timestamped last-write-wins policy; test explicitly in Month 7.
Timeline slippage from coursework/exam overlap.	High	Optional ablation stages (Section 5.5) act as the first items dropped, not core deliverables.
Team member availability/coordination across three concurrent subsystems.	Medium	Clear component ownership (ML / mobile / backend+dashboard) assigned at project start.

Table 5. Key project risks and stated mitigations.

16. REFERENCES

- [1] Moody, G. B., & Mark, R. G. (2001). The impact of the MIT-BIH Arrhythmia Database. *IEEE Engineering in Medicine and Biology Magazine*, 20(3), 45–50.
- [2] Goldberger, A. L., et al. (2000). PhysioBank, PhysioToolkit, and PhysioNet: Components of a New Research Resource for Complex Physiologic Signals. *Circulation*, 101(23), e215–e220.
- [3] Association for the Advancement of Medical Instrumentation (AAMI). Testing and Reporting Performance Results of Cardiac Rhythm and ST Segment Measurement Algorithms (ANSI/AAMI EC57).
- [4] Pan, J., & Tompkins, W. J. (1985). A Real-Time QRS Detection Algorithm. *IEEE Transactions on Biomedical Engineering*, BME-32(3), 230–236.